Android Doctor — Documento de diseño

Nov 6, 2026 · @Marcos Valera Prados

Resumen

Android Doctor es una ventana de editor de Unity, gratuita y de código abierto, que diagnostica en menos de 10 segundos por qué falla (o va a fallar) un build de Android o un envío a Google Play, y dice cómo arreglarlo. Se publica como paquete UPM y como `.unitypackage` en itch.io (pay-what-you-want).

Qué hace, módulo a módulo

- **Entorno:** comprueba que SDK, JDK, NDK y Gradle encajan con tu versión de Unity.
- **Proyecto:** revisa Player Settings, plantillas Gradle y manifest antes de compilar.
- **16 KB:** escanea todas las librerías nativas (`.so` sueltos y dentro de `.aar`) del proyecto y de los paquetes, y dice cuál incumple y a qué paquete pertenece.
- **Build:** tras compilar, abre el APK/AAB y comprueba lo que de verdad se va a subir.
- **Errores de Gradle:** traduce el log de un build fallido a una causa probable y un arreglo.

Qué NO es: no sustituye a Android Studio, no compila librerías nativas, no cambia nada del proyecto sin confirmación y no envía datos a ningún sitio (todo se analiza en local).

Reglas para la IA que lo implemente

1. Trabaja por fases, en el orden de «Plan por fases». No empieces una fase sin haber cerrado la anterior.
2. Cada fase termina con un criterio de aceptación comprobable. Si no se cumple, no avances: arréglalo.
3. No inventes APIs de Unity. Si una API no aparece en este documento o dudas de que exista, búscala en la documentación oficial de la versión de Unity que apuntas, o accede por reflexión y trata el fallo como «desconocido».
4. Los textos de errores de Gradle de este documento son puntos de partida. Cópialos de logs reales a `Tests/Fixtures/` , no de memoria.
5. Un commit por tarea, con el mensaje que indica el plan.
6. Todo texto visible va en un único fichero de mensajes (inglés por defecto, tabla de traducción al español). Nunca dentro de la lógica de las reglas.

Cómo sabremos que funciona (v1.0)

Métrica	Objetivo
Tiempo del primer diagnóstico completo en un proyecto mediano	menos de 10 s
Reglas de entorno y proyecto	al menos 15
Patrones de error de Gradle traducidos	al menos 15, cada uno con un log real de prueba
Falsos positivos de 16 KB en los proyectos de prueba	0
Sistemas operativos	Windows, macOS y Linux
Versiones de Unity	2022.3 LTS, 6000.0 LTS y las 6.x posteriores

El problema

Un build de Android puede fallar por decenas de causas distintas y Unity casi siempre muestra el mismo mensaje: `Gradle build failed`. La causa real queda enterrada en el log de Gradle, y la mayoría de los fallos vienen de que las versiones de SDK, JDK, NDK y Gradle no encajan (fuente). Se nota en los foros: hilos que se repiten año tras año, y gente que paga 25–70 USD en Freelancehunt para que alguien lea su log.

Tres cosas hacen que este dolor crezca justo ahora:

1. **Unity 6.5 cambia el motor de build.** Sube el mínimo de Android a la API 26 y pasa a Gradle 9.1.0 y AGP 9.0.0 (antes 8.11 y 8.7.2). Rompe a quien tenga plantillas Gradle propias o librerías antiguas (Unity: planned breaking changes 6.5).
2. **Google Play exige target API 36.** Las apps nuevas y las actualizaciones deben apuntar a Android 16 (API 36) desde el 31 de agosto de 2026, con una prórroga solicitable hasta el 1 de noviembre de 2026 (Play Console Help). A fecha de este documento (6 de noviembre de 2026) las dos fechas ya han pasado.
3. **Google Play exige 16 KB.** Las librerías nativas de 64 bits deben tener los segmentos ELF `LOAD` alineados a 16 KB (`0x4000`). Hay dos comprobaciones distintas que la gente mezcla: la alineación ELF dentro de cada `.so` (la que valida Play Console) y la alineación ZIP del `.so` dentro del APK (Unity Support). Las versiones de la página de Google que consulté muestran fechas distintas (1 de noviembre de 2025 para apps con target Android 15+ y 1 de febrero de 2027 para actualizaciones). Verifica la fecha vigente en tu Play Console antes de comunicar nada.

Quién lo sufre

- Quien ve `Gradle build failed` por primera vez y no sabe dónde mirar.
- Quien actualiza a Unity 6.x con plantillas Gradle o plugins antiguos.
- Quien sube a Google Play con SDKs de terceros y no sabe cuál de ellos trae la librería mal alineada. Hay hilos con este problema en Firebase y OpenCV, en UnityARCore y Burst y en Vivox.

Lo que existe hoy y dónde se queda corto

Opción actual	Qué hace	Hueco que deja
Plugin Inspector de Unity	Avisa de que un `.so` concreto no está alineado a 16 KB (ejemplo)	Va archivo por archivo, no resume el proyecto ni mira dentro de `.aar` o del build final; hay un caso reportado donde no refrescaba el aviso
APK Analyzer de Android Studio	Analiza el APK ya compilado	Exige instalar Android Studio y puede mostrar un aviso que no coincide con Play Console
`check_elf_alignment.sh` de Google	Comprueba las librerías de un APK	La guía de Google lo ofrece solo para Linux y macOS
CSAF Play Compliance	Herramienta de pago (unos 20 USD según su página; no la he probado)	No es gratis y no lo comprobé a fondo: pruébala antes de empezar para no repetir lo que ya hace bien

Hueco que cubre Android Doctor: una sola herramienta gratuita, que funciona en Windows, macOS y Linux, que junta entorno, proyecto, 16 KB y errores de Gradle, y que dice **a qué paquete pertenece** cada librería mal alineada.

Funcionamiento final esperado

Así se comporta la herramienta terminada, de principio a fin:

1. **Instalar.** `Window > Package Manager > + > Add package from git URL…` con la URL del repositorio (`https://github.com/<usuario>/android-doctor.git?path=/Packages/com.<usuario>.androiddoctor`), o importar el `.unitypackage` de itch.io.

2. **Abrir.** `Window > Android Doctor`. La ventana muestra la versión de Unity, si el módulo de Android está instalado y un botón **Run checks**.
3. **Ejecutar.** Al pulsar, corren todas las reglas con barra de progreso (menos de 10 s en un proyecto mediano). Los resultados se agrupan por categoría y se ordenan por gravedad: error, aviso, info.
4. **Entender.** Cada hallazgo es una tarjeta con título, una frase de explicación, evidencias (rutas, versiones) y, si procede, botones **Fix…**, **Copy command** o **Docs**.
5. **Arreglar.** Los fixes siempre piden confirmación y muestran «antes → después». La herramienta nunca borra archivos, y si edita uno (manifest, plantilla) deja antes una copia `.bak`.
6. **Compilar.** Tras un build de Android correcto, Android Doctor abre el APK/AAB generado y comprueba sus librerías nativas. Si todo está bien escribe una línea en la Consola; si no, un aviso que abre la ventana.
7. **Si el build falla.** En la pestaña **Gradle log**, pulsa **Analyze last build** (lee `Editor.log`) o pega el log. Aparece una tarjeta con la causa probable, el arreglo y el bloque original `What went wrong`.
8. **Compartir. Copy report** copia un informe en Markdown listo para pegar en un foro: versión de Unity, sistema operativo, versiones de JDK/SDK/NDK y hallazgos.

Ejemplo de informe (ilustrativo, los nombres son inventados)

```
## Android Doctor 1.0.0
Unity 6000.3.2f1 · Windows 11 · target API 36 · IL2CPP · ARM64

### Errors (2)
- **NAT001** `libfoo.so` (arm64-v8a) is not 16 KB aligned (LOAD align
0x1000). Owner: `Packages/com.vendor.sdk`. Update the SDK to a 16 KB-
compatible version.
- **ENV004** SDK platform `android-36` is not installed. Run: `sdkmanager
"platforms;android-36"`.

### Warnings (1)
- **PRJ005** Custom Gradle templates found in `Assets/Plugins/Android`.
Review them: Unity 6.5 moves to Gradle 9.1.0 and AGP 9.0.0.
```

Fuera de alcance en v1.0: compilar librerías nativas, instalar SDKs, parchear `.so`, y cualquier llamada a internet.

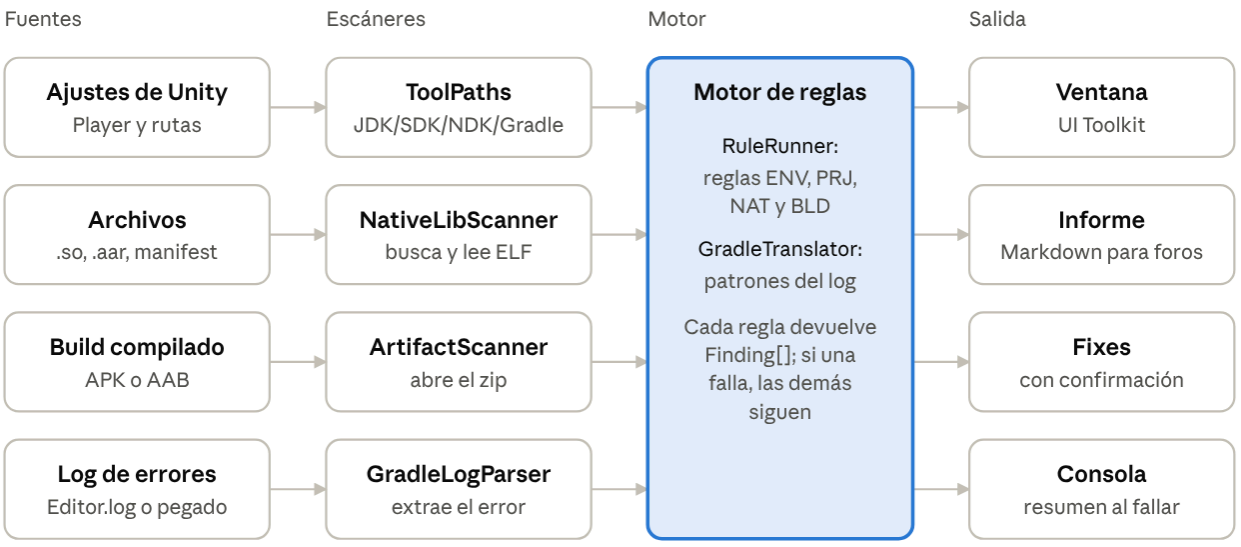
Límites de rendimiento (obligatorios)

- Escanear solo `Assets/` y las carpetas de paquetes resueltos. Ignorar `Library/`, `Temp/`, `Logs/`, `obj/` y `.git/`.

- De cada `.so` leer solo la cabecera ELF y la tabla de program headers con `FileStream` + `Seek`, nunca el archivo entero.
- Solo mirar las ABIs de 64 bits (`arm64-v8a`, `x86_64`) para la comprobación de 16 KB. Las de 32 bits se listan como información y no se juzgan (un hilo de Unity Discussions indica que no lo necesitan).

Arquitectura

Android Doctor convierte cuatro fuentes en hallazgos accionables



Todo se analiza en local: la herramienta no envía datos a ningún sitio.

arquitectura · 4 fuentes, 4 escáneres, 1 motor, 4 salidas

Las fuentes pasan por escáneres que solo leen datos; el motor convierte esos datos en `Finding`; la salida los muestra, los exporta o los arregla.

Reglas de arquitectura (no negociables)

1. `Core` no referencia a `UnityEditor.Android` ni a la UI. Solo conoce `Finding`, `IRule`, `IFix` y `RuleContext`.
2. Los escáneres son clases sin estado estático que devuelven datos. Las reglas los usan a través de `RuleContext`, nunca los llaman directamente.
3. Una regla es una clase con un id estable (`ENV004`). El id nunca se reutiliza ni cambia de significado.
4. Los textos viven en `Messages`; los datos de compatibilidad, en JSON. Ninguno dentro de la lógica.
5. Ninguna regla escribe en disco. Solo un `IFix` puede escribir, y siempre tras confirmación del usuario.

6. Una regla que lanza una excepción no tumba el diagnóstico: el `RuleRunner` la captura y la muestra como un hallazgo informativo «la regla X falló internamente».
7. La UI es UI Toolkit construida desde C# (sin UXML), para que haya menos archivos y menos formas de romperlo.

Paso 0: proyecto, paquete y repositorio

Decisión de estructura: el repositorio raíz es un proyecto Unity de desarrollo y el paquete vive dentro, en `Packages/com.<usuario>.androiddoctor/`. Así se instala desde git con `?path=` y a la vez hay un proyecto donde probarlo. Sustituye `<usuario>` por tu usuario de GitHub (en minúsculas, sin espacios) en todo el documento.

0.1 Requisitos en el PC

- Git y Unity Hub.
- Unity **6000.0 LTS** (última 6000.0.x) con los módulos *Android Build Support*, *OpenJDK* y *Android SDK & NDK Tools*. Es la versión más baja de la línea 6 que apuntamos, así que desarrollar en ella evita usar APIs que no existan allí.
- Opcional pero recomendable para probar: Unity 2022.3 LTS y un editor **sin** el módulo de Android (para comprobar que la herramienta no revienta).
- Un IDE (Visual Studio, Rider o VS Code). Android Studio no es necesario.

0.2 Crear el proyecto

1. Unity Hub > New project > cualquier plantilla 3D (no usamos render) > nombre `android-doctor`.
2. Ubicación **sin espacios ni tildes** (por ejemplo `C:\dev` o `~/dev`). La carpeta resultante `android-doctor` será la raíz del repositorio.
3. `Edit > Project Settings > Editor`: *Version Control Mode* = **Visible Meta Files** y *Asset Serialization* = **Force Text**.
4. `File > Build Profiles` (Unity 6) o `File > Build Settings`: elegir **Android** y *Switch Platform*. Tarda unos minutos.

0.3 Crear `.gitignore` en la raíz

```
[Ll]ibrary/
[Tt]emp/
[Oo]bj/
[Bb]uild/
[Bb]uilds/
[Ll]ogs/
[Uu]ser[Ss]ettings/
[Mm]emoryCaptures/
```

```
.vs/
.idea/
.vscode/
*.csproj
*.sln
*.slnx
*.user
*.suo
*.pidb
*.pdb
*.mdb
*.apk
*.aab
.DS_Store
```

0.4 Crear `.gitattributes` en la raíz

```
* text=auto
*.cs text diff=csharp
*.json text
*.md text
*.so binary
*.aar binary
*.apk binary
*.aab binary
Packages/*/Tests/**/Fixtures/** -text
```

0.5 Crear el paquete

Estructura (crea las carpetas y archivos vacíos desde el explorador de archivos; Unity generará los `.meta`, **y los `.meta` se commitean**):

```
Packages/com.<usuario>.androiddoctor/
  package.json
  README.md
  LICENSE
  CHANGELOG.md
  Editor/
    AndroidDoctor.Editor.asmdef
    Core/
    Environment/
    Project/
    Native/
    Build/
    Gradle/
    UI/
    Data/
  Tests/
    Editor/
      AndroidDoctor.Editor.Tests.asmdef
      Fixtures/
```

`package.json`:

```
{
  "name": "com.<usuario>.androiddoctor",
  "displayName": "Android Doctor",
  "version": "0.1.0",
  "unity": "2022.3",
  "description": "Diagnoses Android build, Gradle and Google Play 16 KB
problems in Unity.",
  "keywords": ["android", "gradle", "google-play", "16kb", "diagnostics"],
  "author": { "name": "<tu nombre>", "url": "https://<tu-usuario>.itch.io" },
  "license": "MIT"
}
```

`Editor/AndroidDoctor.Editor.asmdef`:

```
{
  "name": "AndroidDoctor.Editor",
  "rootNamespace": "AndroidDoctor",
  "references": [],
  "includePlatforms": ["Editor"],
  "autoReferenced": false
}
```

`Tests/Editor/AndroidDoctor.Editor.Tests.asmdef`:

```
{
  "name": "AndroidDoctor.Editor.Tests",
  "rootNamespace": "AndroidDoctor.Tests",
  "references": ["AndroidDoctor.Editor", "UnityEngine.TestRunner",
"UnityEditor.TestRunner"],
  "includePlatforms": ["Editor"],
  "overrideReferences": true,
  "precompiledReferences": ["nunit.framework.dll"],
  "defineConstraints": ["UNITY_INCLUDE_TESTS"],
  "autoReferenced": false
}
```

Ambos `.asmdef` y `package.json` se crean como texto plano y Unity los reconoce al volver al Editor. El `LICENSE` es MIT con tu nombre y el año 2026.

0.6 Primer commit y remoto

Crea antes un repositorio vacío en GitHub (sin README ni licencia) llamado `android-doctor`. Luego, en la raíz del proyecto:

```
git init -b main
git add .
git commit -m "chore: initial Unity project and empty package"
git remote add origin https://github.com/<usuario>/android-doctor.git
git push -u origin main
```

Flujo de ramas y commits (para todo el proyecto)

- `main` siempre compila y pasa los tests.
- Una rama por fase: `feat/fase-1-core`, `feat/fase-2-entorno`, etc. Se fusiona a `main` cuando se cumple el criterio de aceptación de la fase.
- Mensajes en formato Conventional Commits: `feat:`, `fix:`, `test:`, `docs:`, `chore:`.
- Versiones con etiquetas: `v0.1.0` al terminar cada fase; `v1.0.0` al publicar.

Criterio de aceptación de la Fase 0

- El proyecto abre sin errores en la Consola.
- `Window > General > Test Runner` abre y muestra 0 tests, sin errores.
- Existen `package.json` y los dos `.asmdef`, y `git status` está limpio.
- El repositorio está subido a GitHub con el primer commit.

Modelo de datos y motor de reglas

Todo este código va en `Editor/Core/` salvo donde se indique. Es C# 9: **no uses `record` ni `init`** (Unity 2022.3 no trae `IsExternalInit`). Los espacios de nombres son `AndroidDoctor` y `AndroidDoctor.<Módulo>`.

`Severity.cs`

```
namespace AndroidDoctor
{
    public enum Severity { Info = 0, Warning = 1, Error = 2 }
    public enum Category { Environment, Project, Native, Build, Gradle }
}
```

`Finding.cs`: el resultado de cualquier regla. Es inmutable.

```
using System.Collections.Generic;

namespace AndroidDoctor
{
    public sealed class Finding
    {
        public readonly string RuleId;   // "ENV004"
        public readonly Category Category;
        public readonly Severity Severity;
        public readonly string Title;    // una línea
        public readonly string Detail;   // una o dos frases: qué pasa y por
qué importa
        public readonly IReadOnlyList<string> Evidence; // rutas, versiones;
una por línea
        public readonly string FixId;    // null si no hay arreglo automático
        public readonly string FixLabel; // texto del botón
        public readonly string Command;  // comando de consola que se puede
copiar (opcional)
        public readonly string DocsUrl;  // opcional

        public Finding(string ruleId, Category category, Severity severity,
string title, string detail,
            IReadOnlyList<string> evidence = null, string fixId = null,
string fixLabel = null,
            string command = null, string docsUrl = null)
        {
            RuleId = ruleId; Category = category; Severity = severity; Title
= title; Detail = detail;
            Evidence = evidence ?? new string[0];
            FixId = fixId; FixLabel = fixLabel; Command = command; DocsUrl =
```

```
docsUrl;
        }
    }
}
```

`IRule.cs` e `IFix.cs`

```
using System.Collections.Generic;

namespace AndroidDoctor
{
    public interface IRule
    {
        string Id { get; }          // estable para siempre
        Category Category { get; }
        IEnumerable<Finding> Run(RuleContext ctx);
    }

    public interface IFix
    {
        string Id { get; }
        string Describe(Finding finding);                // texto "antes ->
después" para el diálogo
        bool Apply(Finding finding, out string message); // true si se aplicó
    }
}
```

`UnityVersion.cs`

```
using System;
using System.Text.RegularExpressions;

namespace AndroidDoctor
{
    public readonly struct UnityVersion : IComparable<UnityVersion>
    {
        public readonly int Major, Minor, Patch;
        public UnityVersion(int major, int minor, int patch) { Major = major;
Minor = minor; Patch = patch; }

        // Application.unityVersion: "6000.3.2f1", "2022.3.61f1"
        public static UnityVersion Parse(string s)
        {
            var m = Regex.Match(s ?? "", @"^(\d+)\.(\d+)\.(\d+)");
            if (!m.Success) return new UnityVersion(0, 0, 0);
            return new UnityVersion(int.Parse(m.Groups[1].Value),
```

```
int.Parse(m.Groups[2].Value), int.Parse(m.Groups[3].Value));
        }

        public bool AtLeast(int major, int minor) => Major > major || (Major
== major && Minor >= minor);
        public string Key => Major + "." + Minor; // "6000.3"
        public int CompareTo(UnityVersion o)
        {
            int c = Major.CompareTo(o.Major); if (c != 0) return c;
            c = Minor.CompareTo(o.Minor); return c != 0 ? c :
Patch.CompareTo(o.Patch);
        }
        public override string ToString() => Major + "." + Minor + "." +
Patch;
    }
}
```

`AndroidTools.cs` (en `Editor/Environment/`): las rutas de JDK, SDK, NDK y Gradle. Se leen por **reflexión** porque la clase `UnityEditor.Android.AndroidExternalToolsSettings` solo existe si el módulo de Android está instalado y una referencia directa rompería la compilación en editores sin él. Si la API no existe o falla, la ruta queda como `Unknown` y las reglas que dependan de ella se saltan en lugar de dar error. **Verifica los nombres de propiedad (`jdkRootPath`, `sdkRootPath`, `ndkRootPath`, `gradlePath`) en la versión de Unity que uses**; no los he podido comprobar en la documentación oficial.

```
using System;
using System.IO;
using System.Reflection;
using UnityEditor;

namespace AndroidDoctor.Environment
{
    public enum ToolSource { Unknown, Embedded, Custom }

    public sealed class ToolPath
    {
        public string Path;
        public ToolSource Source;
        public bool Exists => !string.IsNullOrEmpty(Path) &&
(Directory.Exists(Path) || File.Exists(Path));
    }

    public sealed class AndroidTools
    {
```

```
        public ToolPath Jdk, Sdk, Ndk, Gradle;
    }

    public static class AndroidToolsReader
    {
        public static AndroidTools Read()
        {
            string engineDir = EngineDir(); //
.../PlaybackEngines/AndroidPlayer (o null)
            return new AndroidTools
            {
                Jdk = Resolve("jdkRootPath", engineDir, "OpenJDK"),
                Sdk = Resolve("sdkRootPath", engineDir, "SDK"),
                Ndk = Resolve("ndkRootPath", engineDir, "NDK"),
                Gradle = Resolve("gradlePath", engineDir,
System.IO.Path.Combine("Tools", "gradle")),
            };
        }

        static string EngineDir()
        {
            try
            {
                string d =
BuildPipeline.GetPlaybackEngineDirectory(BuildTarget.Android,
BuildOptions.None);
                return string.IsNullOrEmpty(d) ? null : d;
            }
            catch { return null; }
        }

        static ToolPath Resolve(string property, string engineDir, string
embeddedSub)
        {
            string embedded = engineDir == null ? null :
System.IO.Path.Combine(engineDir, embeddedSub);
            string configured = ReadSetting(property);
            if (!string.IsNullOrEmpty(configured))
            {
                bool same = embedded != null && string.Equals(
                    System.IO.Path.GetFullPath(configured).TrimEnd('/',
'\\'),
                    System.IO.Path.GetFullPath(embedded).TrimEnd('/', '\\'),
StringComparison.OrdinalIgnoreCase);
                return new ToolPath { Path = configured, Source = same ?
ToolSource.Embedded : ToolSource.Custom };
```

```
            }
            if (embedded != null && Directory.Exists(embedded))
                return new ToolPath { Path = embedded, Source =
ToolSource.Embedded };
            return new ToolPath { Path = null, Source = ToolSource.Unknown };
        }

        static string ReadSetting(string property)
        {
            foreach (var asm in AppDomain.CurrentDomain.GetAssemblies())
            {
                var type =
asm.GetType("UnityEditor.Android.AndroidExternalToolsSettings", false);
                if (type == null) continue;
                var p = type.GetProperty(property, BindingFlags.Public |
BindingFlags.Static);
                if (p == null) return null;
                try { return p.GetValue(null) as string; } catch { return
null; }
            }
            return null;
        }
    }
}
```

Las rutas embebidas (`OpenJDK`, `NDK`, `Tools/gradle`) las confirman rutas reales de logs de errores publicados en el foro de Unity; `SDK` es la convención por simetría: compárala en tu instalación.

`RuleContext.cs`: todo lo que una regla necesita saber, calculado una sola vez por ejecución.

```
using System;
using System.Collections.Generic;
using System.IO;
using AndroidDoctor.Environment;
using UnityEngine;

namespace AndroidDoctor
{
    public sealed class PackageRef
    {
        public string Name, Version, ResolvedPath;
    }

    public sealed class RuleContext
```

```
    {
        public UnityVersion Unity;
        public AndroidTools Tools;
        public CompatTable Compat;      // Editor/Data/compat.json
        public PlayRequirements Play;   // Editor/Data/play.json
        public string ProjectPath;      // carpeta que contiene Assets/
        public string AssetsPath;
        public List<PackageRef> Packages = new List<PackageRef>();

        readonly Dictionary<string, object> _cache = new Dictionary<string,
object>();
        public T GetOrCompute<T>(string key, Func<T> factory)
        {
            if (_cache.TryGetValue(key, out var v)) return (T)v;
            var created = factory();
            _cache[key] = created;
            return created;
        }

        public static RuleContext Create()
        {
            var ctx = new RuleContext
            {
                Unity = UnityVersion.Parse(Application.unityVersion),
                Tools = AndroidToolsReader.Read(),
                ProjectPath =
Directory.GetParent(Application.dataPath).FullName,
                AssetsPath = Application.dataPath,
                Compat = CompatTable.Load(),
                Play = PlayRequirements.Load(),
            };
            foreach (var p in
UnityEditor.PackageManager.PackageInfo.GetAllRegisteredPackages())
                ctx.Packages.Add(new PackageRef { Name = p.name, Version =
p.version, ResolvedPath = p.resolvedPath });
            return ctx;
        }
    }
}
```

Datos en JSON (`Editor/Data/`). `JsonUtility` no lee arrays en la raíz ni diccionarios, así que se usan clases `[Serializable]` con arrays. Un valor `null` significa «desconocido» en las cadenas (en los enteros se usa -1) y la regla que lo necesite se salta.

`compat.json` (solo datos que he verificado en la documentación de Unity; las celdas que no pude confirmar van a `null`; **complétalas leyendo la página «Unity and Gradle version compatibility» de cada versión**):

```
{
  "asOf": "2026-11-06",
  "unity": [
    { "match": "2021.3", "jdk": 11, "ndk": "21.3.6528147", "buildTools":
"34.0.0", "gradle": "6.1.1", "agp": "4.0.1", "minSdk": -1 },
    { "match": "2022.3", "jdk": 11, "ndk": "23.1.7779620", "buildTools":
"34.0.0", "gradle": "7.2", "agp": "7.1.2", "minSdk": -1 },
    { "match": "6000", "jdk": 17, "ndk": "27.2.12479018", "buildTools":
"36.0.0", "gradle": null, "agp": null, "minSdk": -1 },
    { "match": "6000.5", "jdk": 17, "ndk": "27.2.12479018", "buildTools":
"36.0.0", "gradle": "9.1.0", "agp": "9.0.0", "minSdk": 26 }
  ]
}
```

`match` es un prefijo de `Mayor.Menor`; se elige la fila más específica que coincida (`6000.5` gana a `6000`). Para las versiones 6000.6 y posteriores, hereda la fila `6000.5` hasta que la documentación diga otra cosa.

`play.json`:

```
{
  "asOf": "2026-11-06",
  "minTargetSdk": 36,
  "minTargetSdkDeadline": "2026-08-31",
  "require64Bit": true,
  "require16KB": true,
  "knownPackages": [
    { "name": "com.unity.burst", "minVersion": "1.8.21", "severity": "Error",
      "reason": "Older Burst versions ship libburst_generated.so without 16
KB alignment." },
    { "name": "com.google.android.appbundle", "minVersion": "1.10.0",
"severity": "Warning",
      "reason": "Includes bundletool 1.18.1; bundletool 1.17.0+ defaults to
16 KB page alignment (community-reported)." }
  ]
}
```

Las dos clases que los cargan (`CompatTable`, `PlayRequirements`) siguen el mismo patrón: clase `[Serializable]` con campos públicos que reflejan el JSON, y un `static Load()` que localiza el paquete con `UnityEditor.PackageManager.PackageInfo.FindForAssembly(typeof(CompatTable).As`

`sembly).resolvedPath`, lee `Editor/Data/<archivo>.json` con `File.ReadAllText` y llama a `JsonUtility.FromJson`. Los campos que pueden ser `null` en el JSON se declaran como `string` (las cadenas sí admiten `null`) o como `int` con el valor `-1` para «desconocido»: `JsonUtility` no admite `int?`. `CompatTable` expone `Find(UnityVersion v)` que devuelve la fila más específica o `null`.

`RuleRunner.cs`: ejecuta, captura fallos y ordena.

```
using System;
using System.Collections.Generic;
using System.Linq;
using UnityEditor;

namespace AndroidDoctor
{
    public static class RuleRunner
    {
        public static List<IRule> DiscoverRules()
        {
            var rules = new List<IRule>();
            foreach (var t in TypeCache.GetTypesDerivedFrom<IRule>())
            {
                if (t.IsAbstract || t.IsInterface ||
t.GetConstructor(Type.EmptyTypes) == null) continue;
                rules.Add((IRule)Activator.CreateInstance(t));
            }
            return rules.OrderBy(r => r.Id, StringComparer.Ordinal).ToList();
        }

        public static List<Finding> RunAll(IList<IRule> rules, RuleContext
ctx, Action<float, string> progress = null)
        {
            var all = new List<Finding>();
            for (int i = 0; i < rules.Count; i++)
            {
                var rule = rules[i];
                progress?.Invoke((float)i / rules.Count, rule.Id);
                try
                {
                    var found = rule.Run(ctx);
                    if (found != null) all.AddRange(found);
                }
                catch (Exception e)
                {
                    all.Add(new Finding(rule.Id, rule.Category,
Severity.Info,
```

```
                        "Rule " + rule.Id + " failed internally",
e.GetType().Name + ": " + e.Message));
                }
            }
            return all.OrderByDescending(f => f.Severity).ThenBy(f =>
f.Category).ThenBy(f => f.RuleId).ToList();
        }
    }
}
```

`FixRegistry` hace lo mismo con `IFix` (`TypeCache.GetTypesDerivedFrom<IFix>()`) y expone `Get(string fixId)`.

`Messages.cs`: todo texto visible pasa por aquí. Inglés por defecto; español si `Application.systemLanguage == SystemLanguage.Spanish`; si falta una clave en español, se usa la inglesa; si falta también, se devuelve la propia clave (así un olvido se ve en pantalla y no revienta).

```
using System.Collections.Generic;
using UnityEngine;

namespace AndroidDoctor
{
    public static class Messages
    {
        static readonly Dictionary<string, string> En = new
Dictionary<string, string>
        {
            { "PRJ009.title", "Default application identifier" },
            { "PRJ009.detail", "The identifier starts with
com.DefaultCompany. You cannot change it after publishing to Google Play." },
        };
        static readonly Dictionary<string, string> Es = new
Dictionary<string, string>
        {
            { "PRJ009.title", "Identificador de aplicación por defecto" },
            { "PRJ009.detail", "El identificador empieza por
com.DefaultCompany. No se puede cambiar después de publicar en Google Play."
},
        };

        public static string Get(string key, params object[] args)
        {
            string s;
            bool es = Application.systemLanguage == SystemLanguage.Spanish;
            if (!(es && Es.TryGetValue(key, out s)) && !En.TryGetValue(key,
```

```
out s)) s = key;
            return args == null || args.Length == 0 ? s : string.Format(s,
args);
        }
    }
}
```

Ejemplo completo de una regla (`Editor/Project/DefaultApplicationIdRule.cs`). Toda regla nueva se escribe con esta forma: una clase, un id, `yield return` por cada hallazgo, textos en `Messages`.

```
using System.Collections.Generic;
using UnityEditor;
using UnityEditor.Build;

namespace AndroidDoctor.Project
{
    public sealed class DefaultApplicationIdRule : IRule
    {
        public string Id => "PRJ009";
        public Category Category => Category.Project;

        public IEnumerable<Finding> Run(RuleContext ctx)
        {
            string id =
PlayerSettings.GetApplicationIdentifier(NamedBuildTarget.Android);
            if (string.IsNullOrEmpty(id) ||
id.StartsWith("com.DefaultCompany."))
            {
                yield return new Finding(Id, Category, Severity.Warning,
                    Messages.Get("PRJ009.title"),
Messages.Get("PRJ009.detail"),
                    new[] { "Application identifier: " + id });
            }
        }
    }
}
```

Catálogo de reglas

Estas son todas las reglas de v1.0, por orden de implementación. Un id nunca se reutiliza. **Severidad:** E = Error, W = Warning, I = Info. Las columnas «Cómo» nombran APIs y archivos concretos; si una API de Unity no existe en la versión que apuntas, la regla debe devolver sin hallazgos y no fallar.

Id	Qué comprueba	Cómo	Sev.	Arreglo que ofrece
ENV001	Módulo de Android instalado	`BuildPipeline.IsBuildTargetSupported(BuildTargetGroup.Android, BuildTarget.Android)`	E	Texto: Unity Hub > Installs > Add modules
ENV002	JDK existe y su versión encaja con Unity	Ruta de `AndroidToolsReader`; leer el archivo `<jdk>/release` (línea `JAVA_VERSION="17.0.x"`); comparar con `compat.json.jdk`	E si la ruta no existe; W si es un JDK propio de otra versión mayor	Botón que abre `Preferences/External Tools` con `SettingsService.OpenUserPreferences`; texto: marcar «JDK Installed with Unity (recommended)»
ENV003	SDK existe y contiene `platform-tools` y `cmdline-tools` (o `tools`)	Carpetas dentro de `Sdk.Path`	E	Texto: usar el SDK que instala Unity
ENV004	SDK Platform de la API objetivo instalada	Carpeta `<sdk>/platforms/android-N`, con N = target API efectivo (si es Auto, la plataforma más alta instalada)	E	Copiar `sdkmanager "platforms;android-N"`
ENV005	Build-tools instaladas (informativo)	Carpetas `<sdk>/build-tools/*` frente a `compat.json.buildTools`. La documentación de Unity dice que no exige una versión exacta de SDK	I	Ninguno
ENV006	NDK existe y su versión encaja (solo con IL2CPP)	`<ndk>/source.properties`, clave `Pkg.Revision`; comparar con `compat.json.ndk`	E si falta; W si es distinta	Texto: usar el NDK que instala Unity
ENV007	Gradle propio compatible	Versión del archivo `lib/gradle-launcher-X.Y.jar` de la carpeta Gradle; comparar con `compat.json.gradle` (si no es `null`)	W	Texto: usar el Gradle de Unity
ENV008	Variables de entorno que interfieren con Gradle	`Environment.GetEnvironmentVariable` de `ANDROID_SDK_HOME`, `ANDROID_HOME`, `JAVA_HOME`, `ANDROID_NDK_HOME`, `GRADLE_USER_HOME`	W si `ANDROID_SDK_HOME` está definida (aparece en errores reales de Unity); I para las demás	Texto: quitar o corregir la variable y reiniciar Unity Hub
ENV009	Ruta del proyecto problemática	Windows y ruta de más de 200 caracteres (W); caracteres no ASCII (I). Heurística, sin fuente oficial	W/I	Texto: mover el proyecto a una ruta corta
PRJ001	Target API ≥ mínimo de Google Play	`PlayerSettings.Android.targetSdkVersion` frente a `play.json.minTargetSdk`; si es `AndroidApiLevelAuto`, la plataforma más alta instalada	E	Fix: poner la API mínima (con confirmación)

Id	Qué comprueba	Cómo	Sev.	Arreglo que ofrece
PRJ002	Min API ≥ mínimo de Unity	`PlayerSettings.Android.minSdkVersion` frente a `compat.json.minSdk` (si no es -1)	E	Fix: subir la Min API
PRJ003	ARM64 activado y backend IL2CPP	`PlayerSettings.Android.targetArchitectures` y `PlayerSettings.GetScriptingBackend(NamedBuildTarget.Android)`	E si solo hay ARMv7	Fix: activar ARM64 y IL2CPP (con confirmación)
PRJ004	Android App Bundle activado (informativo)	`EditorUserBuildSettings.buildAppBundle`	I	Ninguno
PRJ005	Plantillas Gradle personalizadas	Existencia de `mainTemplate.gradle`, `launcherTemplate.gradle`, `baseProjectTemplate.gradle`, `settingsTemplate.gradle` y `gradleTemplate.properties` en `Assets/Plugins/Android/`	W si Unity ≥ 6.5, I si no	Texto: comparar con las plantillas por defecto de `PlaybackEngines/AndroidPlayer/Tools/GradleTemplates/`
PRJ006	Componentes del manifest sin `android:exported`	Parsear con `System.Xml.Linq` los `AndroidManifest.xml` de `Assets/Plugins/Android/` y de las carpetas `*.androidlib`; buscar `activity`, `service` y `receiver` con `intent-filter` y sin el atributo	E si target API ≥ 31	Mostrar la línea exacta que añadir; no editar solo
PRJ007	Librerías `.aar` / `.jar` duplicadas con versiones distintas	Agrupar por nombre de artefacto (`nombre-1.2.3.aar` → `nombre`) dentro de `Assets/`	W	Botón «Select in Project»; texto: quitar la antigua o ejecutar el resolvedor de EDM4U
PRJ008	Release firmada con la clave de debug	`PlayerSettings.Android.useCustomKeystore` y `EditorUserBuildSettings.development`	W	Texto: Player Settings > Publishing Settings
PRJ009	Application ID por defecto	`PlayerSettings.GetApplicationIdentifier` empieza por `com.DefaultCompany.`	W	Texto: cambiarlo antes de publicar
NAT001	`.so` de 64 bits con segmento `LOAD` sin alinear a 16 KB	`ElfReader` sobre `.so` de `Assets/`, de los paquetes y de dentro de los `.aar` (`jni/<abi>/`)	E	Mostrar el paquete dueño; texto: actualizar a una versión compatible o contactar al proveedor
NAT002	Segmento RELRO no alineado a 16 KB	En `PT_GNU_RELRO`, (`p_vaddr` + `p_memsz`) mod 0x4000 distinto de 0 (criterio de la guía de Google)	W	Texto: recompilar la librería
NAT003	Librería con ABI de 32 bits y sin la de 64 bits	Comparar, por nombre de `.so` y carpeta de plugin, las ABIs presentes	E	Texto: pedir la versión de 64 bits

Id	Qué comprueba	Cómo	Sev.	Arreglo que ofrece
NAT004	Burst demasiado antiguo	Versión de `com.unity.burst` < `play.json.knownPackages` (1.8.21, según Unity)	E	Texto: actualizar en Package Manager
NAT005	`com.google.android.appbundle` antiguo	Versión < 1.10.0; **dato aportado por un usuario en un foro**, no oficial	W	Texto: actualizar el paquete
BLD001	`.so` del APK/AAB compilado con `LOAD` sin alinear	`ArtifactScanner` abre el zip: `lib/<abi>/*.so` en un APK, `*/lib/<abi>/*.so` en un AAB	E	Igual que NAT001, atribuyendo por nombre de archivo
BLD002	Alineación ZIP de los `.so` sin comprimir (solo APK; **fase 9, opcional**)	Posición del dato de cada entrada en el zip módulo 16384, como hace `zipalign -c -P 16 4`	I	Texto: es secundaria; Play Console es la referencia

Atribución (la ventaja principal de NAT001 y BLD001): cada `.so` encontrado en el proyecto se guarda con su *dueño*: el paquete UPM (`com.vendor.sdk`), la carpeta de plugin dentro de `Assets/` o el `.aar` que lo contiene. En BLD001, un `.so` malo del build se enlaza con el escaneo del proyecto por **nombre de archivo y ABI**. Si no hay coincidencia, el hallazgo dice: «no está en tu proyecto; probablemente llega de una dependencia transitiva de Gradle o de Unity». No inventes un dueño.

Escáner de alineación de 16 KB

Qué se comprueba. Play Console valida la alineación **ELF** de cada librería nativa de 64 bits: ningún segmento `LOAD` puede tener `p_align` menor que 0x4000 (2^14), según la guía de Google. La alineación ZIP del `.so` dentro del APK es un requisito distinto y secundario (Unity Support); queda para la fase 9.

Campos del formato ELF que se leen (todo little-endian; Android no usa big-endian)

Campo	ELF32	ELF64
Marca `0x7F 'E' 'L' 'F'`	bytes 0–3	bytes 0–3
Clase (1 = 32 bits, 2 = 64 bits)	byte 4	byte 4
Endianness (1 = little)	byte 5	byte 5
`e_machine` (2 bytes)	0x12	0x12
`e_phoff` (inicio de la tabla de program headers)	0x1C (4 bytes)	0x20 (8 bytes)
`e_phentsize` (2 bytes)	0x2A	0x36
`e_phnum` (2 bytes)	0x2C	0x38

Campo	ELF32	ELF64
Program header: `p_type`	+0x00	+0x00
`p_offset` / `p_vaddr`	+0x04 / +0x08	+0x08 / +0x10
`p_filesz` / `p_memsz`	+0x10 / +0x14	+0x20 / +0x28
`p_align`	+0x1C	+0x30

Constantes: `PT_LOAD = 1`, `PT_GNU_RELRO = 0x6474E552`, `EM_AARCH64 = 0xB7`, `EM_X86_64 = 0x3E`, `EM_ARM = 0x28`, `EM_386 = 0x03`.

`Native/ElfReader.cs`

```
using System.Collections.Generic;
using System.IO;

namespace AndroidDoctor.Native
{
    public struct ElfSegment { public ulong Offset, VAddr, FileSize, MemSize,
Align; }

    public sealed class ElfInfo
    {
        public bool Is64;
        public ushort Machine;
        public readonly List<ElfSegment> Loads = new List<ElfSegment>();
        public ElfSegment? Relro;

        public ulong MinLoadAlign()
        {
            if (Loads.Count == 0) return 0;
            ulong min = ulong.MaxValue;
            foreach (var l in Loads) if (l.Align < min) min = l.Align;
            return min;
        }
        public bool Is16KbLoadAligned => Loads.Count > 0 && MinLoadAlign() >=
0x4000;
        public bool IsRelroAligned16Kb => !Relro.HasValue ||
((Relro.Value.VAddr + Relro.Value.MemSize) % 0x4000UL) == 0;
    }

    public static class ElfReader
    {
        const uint PT_LOAD = 1, PT_GNU_RELRO = 0x6474E552;
```

```
        public static bool TryRead(Stream s, out ElfInfo info, out string
error)
        {
            info = null; error = null;
            try
            {
                var r = new BinaryReader(s); // BinaryReader lee little-
endian
                s.Seek(0, SeekOrigin.Begin);
                byte[] id = r.ReadBytes(16);
                if (id.Length < 16 || id[0] != 0x7F || id[1] != (byte)'E' ||
id[2] != (byte)'L' || id[3] != (byte)'F')
                { error = "not an ELF file"; return false; }
                if (id[5] != 1) { error = "big-endian ELF is not supported";
return false; }
                bool is64 = id[4] == 2;
                if (id[4] != 1 && !is64) { error = "unknown ELF class";
return false; }

                s.Seek(0x12, SeekOrigin.Begin);
                ushort machine = r.ReadUInt16();

                ulong phoff; ushort phentsize, phnum;
                if (is64)
                {
                    s.Seek(0x20, SeekOrigin.Begin); phoff = r.ReadUInt64();
                    s.Seek(0x36, SeekOrigin.Begin); phentsize =
r.ReadUInt16(); phnum = r.ReadUInt16();
                }
                else
                {
                    s.Seek(0x1C, SeekOrigin.Begin); phoff = r.ReadUInt32();
                    s.Seek(0x2A, SeekOrigin.Begin); phentsize =
r.ReadUInt16(); phnum = r.ReadUInt16();
                }

                var result = new ElfInfo { Is64 = is64, Machine = machine };
                for (int i = 0; i < phnum; i++)
                {
                    s.Seek((long)phoff + (long)i * phentsize,
SeekOrigin.Begin);
                    uint type = r.ReadUInt32();
                    var seg = new ElfSegment();
                    if (is64)
                    {
                        r.ReadUInt32();                                    //
```

```
p_flags
                        seg.Offset = r.ReadUInt64(); seg.VAddr =
r.ReadUInt64();
                        r.ReadUInt64();                                  //
p_paddr
                        seg.FileSize = r.ReadUInt64(); seg.MemSize =
r.ReadUInt64(); seg.Align = r.ReadUInt64();
                    }
                    else
                    {
                        seg.Offset = r.ReadUInt32(); seg.VAddr =
r.ReadUInt32();
                        r.ReadUInt32();                                  //
p_paddr
                        seg.FileSize = r.ReadUInt32(); seg.MemSize =
r.ReadUInt32();
                        r.ReadUInt32();                                  //
p_flags
                        seg.Align = r.ReadUInt32();
                    }
                    if (type == PT_LOAD) result.Loads.Add(seg);
                    else if (type == PT_GNU_RELRO) result.Relro = seg;
                }
                info = result;
                return true;
            }
            catch (EndOfStreamException) { error = "truncated ELF file";
return false; }
            catch (IOException e) { error = e.Message; return false; }
        }
    }
}
```

`Native/NativeLibScanner.cs`: encuentra y lee las librerías del proyecto.

1. Raíces a recorrer: `ctx.AssetsPath` y el `ResolvedPath` de cada paquete de `ctx.Packages`.
2. Recorrer con una recursión propia que rellena una `List<string>` (no uses `yield` dentro de `try/catch`, no compila). Saltar carpetas que empiecen por `.`, que terminen en `~` (Unity las ignora) o se llamen `Library`, `Temp`, `obj`, `Logs`, `node_modules`. Ignorar errores de acceso.
3. De cada archivo `*.so` o `*.aar`, **comprobar si va a Android**: obtener su ruta de asset (`Assets/...` o `Packages/<nombre-del-paquete>/<ruta-relativa>`), pedir `AssetImporter.GetAtPath(assetPath) as PluginImporter` y quedarse solo con los

que cumplan `GetCompatibleWithPlatform(BuildTarget.Android)`. Si el importer es `null`, ignorar el archivo. **Esto evita el falso positivo más probable**: los `.so` de plugins de Linux de escritorio (`Plugins/x86_64/libfoo.so`) no son librerías de Android.

4. ABI de un `.so` suelto: el primer segmento de la ruta que sea `arm64-v8a`, `armeabi-v7a`, `x86_64`, `x86` o `armeabi`. Si no hay ninguno, deducirla de `e_machine` (`0xB7` → `arm64-v8a`, `0x3E` → `x86_64`, `0x28` → `armeabi-v7a`, `0x03` → `x86`).
5. De un `.aar` (es un zip): abrirlo con `new ZipArchive(File.OpenRead(ruta), ZipArchiveMode.Read)`, y leer las entradas `jni/<abi>/*.so`. Su ubicación se escribe `ruta.aar!/jni/arm64-v8a/libfoo.so` y su dueño es el nombre del `.aar`.
6. **Dueño** de un `.so` suelto: el nombre del paquete UPM si está en un paquete; si está en `Assets/`, la primera carpeta de su ruta que **no** sea genérica (`Plugins`, `Android`, `libs`, `lib`, `jni`) y antes de la carpeta de ABI; si todas son genéricas, la carpeta que contiene la carpeta de ABI.

```
public sealed class NativeLib
{
    public string Name, Abi, Location, Owner, ReadError;
    public ElfInfo Elf;          // null si no se pudo leer
    public bool Is64Bit => Abi == "arm64-v8a" || Abi == "x86_64";
}
```

El resultado se guarda con `ctx.GetOrCompute("nativeLibs", () => NativeLibScanner.Scan(ctx))` para que NAT001, NAT002 y NAT003 compartan una única pasada.

`Build/ArtifactScanner.cs`: lee el APK o AAB ya compilado.

```
using System.Collections.Generic;
using System.IO;
using System.IO.Compression;
using System.Text.RegularExpressions;
using AndroidDoctor.Native;

namespace AndroidDoctor.Build
{
    public static class ArtifactScanner
    {
        // APK: lib/<abi>/x.so    AAB: base/lib/<abi>/x.so (o
<modulo>/lib/<abi>/x.so)
        static readonly Regex LibEntry = new Regex(@"^(?:[^/]+/)?lib/(arm64-
v8a|x86_64|armeabi-v7a|x86)/([^/]+\.so)$");

        public static List<NativeLib> Scan(string apkOrAabPath)
        {
```

```
            var list = new List<NativeLib>();
            using (var fs = new FileStream(apkOrAabPath, FileMode.Open,
FileAccess.Read, FileShare.Read))
            using (var zip = new ZipArchive(fs, ZipArchiveMode.Read))
            {
                foreach (var e in zip.Entries)
                {
                    var m = LibEntry.Match(e.FullName);
                    if (!m.Success) continue;
                    var lib = new NativeLib
                    {
                        Name = m.Groups[2].Value, Abi = m.Groups[1].Value,
                        Location = apkOrAabPath + "!/" + e.FullName, Owner =
"(build output)"
                    };
                    string error;
                    lib.Elf = ReadElf(e, out error);
                    lib.ReadError = lib.Elf == null ? error : null;
                    list.Add(lib);
                }
            }
            return list;
        }

        // Primero los 64 KB iniciales (la tabla de program headers suele
estar justo tras la cabecera);
        // si no cabe, la entrada completa.
        static ElfInfo ReadElf(ZipArchiveEntry e, out string error)
        {
            ElfInfo info;
            foreach (long limit in new long[] { 65536, long.MaxValue })
            {
                using (var s = e.Open())
                using (var ms = new MemoryStream())
                {
                    CopyUpTo(s, ms, limit);
                    if (ElfReader.TryRead(ms, out info, out error)) return
info;
                    if (limit == long.MaxValue) return null;
                }
            }
            error = "unreachable";
            return null;
        }

        static void CopyUpTo(Stream src, Stream dst, long max)
```

```
        {
            var buf = new byte[81920]; long total = 0; int n;
            while (total < max && (n = src.Read(buf, 0,
(int)System.Math.Min(buf.Length, max - total))) > 0)
            { dst.Write(buf, 0, n); total += n; }
            dst.Position = 0;
        }
    }
}
```

Enganche tras el build (`Build/PostBuildScan.cs`): una clase `IPostprocessBuildWithReport` con `callbackOrder => 1000`. En `OnPostprocessBuild(BuildReport report)`: salir si `report.summary.platform != BuildTarget.Android`; tomar `report.summary.outputPath`; si `File.Exists(...)` es falso (es una exportación de proyecto Gradle, una carpeta) salir; escanear con `ArtifactScanner`; si hay librerías de 64 bits desalineadas, `Debug.LogWarning("Android Doctor: N native libraries in your build are not 16 KB aligned. Open Window > Android Doctor.")`; si no, `Debug.Log("Android Doctor: build checked, no 16 KB problems.")`. Guarda la ruta en `SessionState.SetString("AndroidDoctor.LastArtifact", ruta)` para que la ventana pueda reanalizarlo.

Tabla de decisión (qué se convierte en hallazgo)

Situación	Resultado
Librería de 64 bits y `MinLoadAlign() >= 0x4000`	Nada
Librería de 64 bits y `MinLoadAlign() < 0x4000`	Error NAT001 (o BLD001 si viene del build), agrupado por dueño
Librería de 64 bits y RELRO desalineado	Warning NAT002
Archivo ilegible o que no es ELF	Info «no se pudo leer X: motivo». Nunca Error
Librerías de 32 bits	Una sola línea Info: «N librerías de 32 bits no se evalúan»
`.so` que el `PluginImporter` no marca como Android	Se ignora sin decir nada

Un hallazgo agrupado por dueño tiene el título «<N> librerías de <dueño> no están alineadas a 16 KB» y una línea de evidencia por librería: `libfoo.so (arm64-v8a) · LOAD align 0x1000 · <ubicación>`. Si el dueño es un paquete UPM, el detalle añade su versión actual y la frase «actualiza el paquete o pide al proveedor una versión con soporte de 16 KB».

Traductor de errores de Gradle

Objetivo: convertir el muro de texto de un build fallido en «causa probable + arreglo», y mostrar siempre el bloque original por si no hay coincidencia.

Entradas (las tres alimentan el mismo analizador)

1. **Pegar** el log en un cuadro de texto de la pestaña *Gradle log*.
2. **Analyze last build:** leer `Editor.log`. Las rutas están en el manual de Unity: Windows `%LOCALAPPDATA%\Unity\Editor\Editor.log`, macOS `~/Library/Logs/Unity/Editor.log`, Linux `~/.config/unity3d/Editor.log`. Unity lo tiene abierto mientras corre, así que hay que abrirlo con `FileShare.ReadWrite`.
3. **Captura automática:** un `[InitializeOnLoad]` que se suscribe a `Application.logMessageReceived` (el que corre en el hilo principal, **no** el `Threaded`, porque `SessionState` solo se puede usar en el hilo principal). Si el mensaje es `Error` o `Exception` y contiene `Gradle build failed`, guarda el texto con `SessionState.SetString("AndroidDoctor.LastGradleError", texto)` y escribe un `Debug.LogWarning` con la causa probable. El mensaje de la consola puede venir recortado, por eso `Editor.log` es la fuente completa.

```
namespace AndroidDoctor.Gradle
{
    public static class LogSource
    {
        public static string EditorLogPath()
        {
#if UNITY_EDITOR_WIN
            return System.IO.Path.Combine(System.Environment.GetFolderPath(
                System.Environment.SpecialFolder.LocalApplicationData),
"Unity", "Editor", "Editor.log");
#elif UNITY_EDITOR_OSX
            return System.IO.Path.Combine(System.Environment.GetFolderPath(
                System.Environment.SpecialFolder.Personal), "Library",
"Logs", "Unity", "Editor.log");
#else
            return System.IO.Path.Combine(System.Environment.GetFolderPath(
                System.Environment.SpecialFolder.Personal), ".config",
```

```
"unity3d", "Editor.log");
#endif
        }

        // Lee solo el final del archivo (por defecto 2 MB).
        public static string ReadTail(string path, int maxBytes = 2000000)
        {
            using (var fs = new System.IO.FileStream(path,
System.IO.FileMode.Open,
                System.IO.FileAccess.Read, System.IO.FileShare.ReadWrite))
            {
                long start = System.Math.Max(0, fs.Length - maxBytes);
                fs.Seek(start, System.IO.SeekOrigin.Begin);
                using (var sr = new System.IO.StreamReader(fs)) return
sr.ReadToEnd();
            }
        }
    }
}
```

`GradleLogParser.Extract(string log)` devuelve un `GradleError { string Block; string WhatWentWrong; }`:

1. Buscar la **última** aparición de `FAILURE: Build failed with an exception`. Gradle imprime a continuación secciones que empiezan por `* Where:`, `* What went wrong:` y `* Try:` (así aparecen en logs reales).
2. `Block` = desde esa línea hasta la línea `BUILD FAILED` (o el final del texto).
3. `WhatWentWrong` = el texto entre `* What went wrong:` y la siguiente línea que empiece por `* Try:` o `* Exception is:`.
4. Si no existe `FAILURE: Build failed`, pero sí `CommandInvokationFailure: Gradle build failed`, tomar las 60 líneas siguientes como `Block`. Si no hay ninguna de las dos, devolver `null` y la ventana dice «no encuentro un fallo de Gradle en este texto».

`GradleTranslator.Translate(GradleError e, RuleContext ctx)` ejecuta cada patrón de la lista sobre `Block`, genera un `Finding` por coincidencia (los textos con huecos `{0}` se rellenan con los grupos capturados) y los ordena por `Priority`. **Si ningún patrón coincide**, devuelve un único hallazgo `GRD000`: título «Gradle falló y no reconozco el error», la evidencia es `WhatWentWrong` y un botón **Copy for forum** copia el bloque con la versión de Unity. Eso ya es útil por sí solo: es lo que hay que pegar al pedir ayuda.

Patrones de v1.0 (cada uno con al menos un log real en `Tests/Editor/Fixtures/Gradle/`)

Id	Qué significa	Qué se le dice al usuario
GRD001	El JDK es más nuevo de lo que entiende el Gradle usado («Unsupported class file major version N»; la versión de Java es N − 44)	Usar el JDK que instala Unity (`Preferences > External Tools`)
GRD002	Licencias del SDK sin aceptar	Ejecutar `sdkmanager --licenses` (el comando se calcula con la ruta del SDK y se copia con un clic)
GRD003	Falta la plataforma del SDK de la API N	`sdkmanager "platforms;android-N"`, o bajar el target API
GRD004	Faltan las build-tools de una revisión	`sdkmanager "build-tools;X"`
GRD005	NDK ausente o incompleto	Usar el NDK que instala Unity
GRD006	Clase duplicada en dos módulos (`Duplicate class`)	Hay dos librerías con las mismas clases. Ejecutar PRJ007 y quitar la versión antigua; si usas EDM4U, relanzar su resolvedor
GRD007	Un componente del manifest no declara `android:exported`	Añadir `android:exported="true"` o `"false"` en ese elemento (ver PRJ006)
GRD008	Dos plugins traen el mismo `.so` («N files found with path 'lib/…so'»)	Quitar uno de los dos o añadir una regla `pickFirst` para esa ruta en `mainTemplate.gradle` (la sintaxis depende de tu AGP; enlazar a la documentación de Android)
GRD009	Gradle se queda sin memoria	Subir el heap en `org.gradle.jvmargs` de `gradleTemplate.properties` (plantilla de propiedades de Gradle personalizada)
GRD010	Problema de red al descargar dependencias	Comprobar conexión, VPN, proxy o certificados corporativos
GRD011	Una librería exige una Min API mayor que la del proyecto	Subir la Min API a N o usar otra versión de la librería
GRD012	Demasiados métodos para un solo dex	Activar minificación/multidex según la versión; es raro con Min API 21 o más

Id	Qué significa	Qué se le dice al usuario
GRD013	El archivo supera 4 GB (`Archive's size exceeds the limit of 4GByte`, visto en foros de Unity)	Reducir el tamaño o usar AAB con Play Asset Delivery
GRD014	Una plantilla Gradle personalizada tiene un error de sintaxis («Could not compile build file …», visto en un hilo real)	Comparar con la plantilla por defecto o borrar la plantilla para probar
GRD015	`ANDROID_SDK_HOME` definida y fallo al aplicar el plugin de Android (visto en un log real)	Quitar o corregir esa variable de entorno (ver ENV008)
GRD016	Contraseña o alias de la keystore incorrectos	Revisar `Player Settings > Publishing Settings`
GRD017	El JDK es más viejo de lo que exige el plugin de Android («requires Java N to run»)	Usar el JDK que instala Unity

`Gradle/GradlePatterns.cs` (datos; estas expresiones son un **punto de partida**: valida cada una contra un log real y ajusta el texto, no las des por buenas)

```
using System.Text.RegularExpressions;

namespace AndroidDoctor.Gradle
{
    public sealed class GradlePattern
    {
        public string Id;       // "GRD001"
        public int Priority;    // menor = se muestra antes
        public Regex Rx;
        public GradlePattern(string id, int priority, string regex)
        { Id = id; Priority = priority; Rx = new Regex(regex,
RegexOptions.IgnoreCase | RegexOptions.Multiline); }
    }

    public static class GradlePatterns
    {
        public static readonly GradlePattern[] All =
        {
            new GradlePattern("GRD001", 10, @"Unsupported class file major
version (\d+)"),
            new GradlePattern("GRD002", 10, @"licen[cs]es? .*have not been
accepted|Failed to install the following Android SDK packages as some
```

```
licen[cs]es"),
            new GradlePattern("GRD003", 10, @"Failed to find target with hash
string 'android-(\d+)'|Failed to find Platform SDK with path:
platforms;android-(\d+)"),
            new GradlePattern("GRD004", 10, @"Failed to find Build Tools
revision ([\d.]+)"),
            new GradlePattern("GRD005", 10, @"NDK at .* did not have a
source\.properties|No version of NDK matched the requested version|NDK is not
installed"),
            new GradlePattern("GRD006", 20, @"Duplicate class ([\w.$]+) found
in modules"),
            new GradlePattern("GRD007", 20, @"android:exported needs to be
explicitly specified for element <(\w+)"),
            new GradlePattern("GRD008", 20, @"(\d+) files found with path
'(lib/[^']+\.so)'"),
            new GradlePattern("GRD009", 30,
@"java\.lang\.OutOfMemoryError|Java heap space|GC overhead limit
exceeded|Expiring Daemon because JVM heap space is exhausted"),
            new GradlePattern("GRD010", 40, @"UnknownHostException|Connect
timed out|PKIX path building failed|Could not (?:GET|HEAD) '?https?://"),
            new GradlePattern("GRD011", 20, @"minSdkVersion (\d+) cannot be
smaller than version (\d+) declared in library \[?([^\]\s]+)"),
            new GradlePattern("GRD012", 40, @"Cannot fit requested classes in
a single dex file|method ID not in \[0, 0xffff\]"),
            new GradlePattern("GRD013", 20, @"Archive's size exceeds the
limit of 4GByte|enable the zip64 extension"),
            new GradlePattern("GRD014", 15, @"Could not compile build file
'([^']+)'"),
            new GradlePattern("GRD015", 5,  @"ANDROID_SDK_HOME is set to the
root of your SDK"),
            new GradlePattern("GRD016", 20, @"Keystore was tampered with, or
password was incorrect|Failed to read key .* from store"),
            new GradlePattern("GRD017", 10, @"Android Gradle plugin requires
Java (\d+) to run\. You are currently using Java (\d+)"),
        };
    }
}
```

Reglas de calidad del traductor

- Un patrón sin log real de prueba no se publica.
- El texto de cada arreglo dice qué hacer y dónde, en una o dos frases, sin jerga de Gradle.
- La traducción **nunca oculta** el bloque original: va siempre debajo, plegado.

- Si dos patrones coinciden, se muestran ambos ordenados por `Priority` (ej.: GRD015 va antes que cualquier otro porque explica el fallo de raíz).

Interfaz

Tecnología: UI Toolkit construida desde C# (`UnityEngine.UIElements`, `UnityEditor.UIElements`), sin UXML ni USS. En Unity 2022.3 no existe `TabView`, así que las pestañas se hacen con dos paneles y `style.display`.

Estructura de `UI/AndroidDoctorWindow.cs`

```
Window > Android Doctor            [MenuItem("Window/Android Doctor")]
├─ Cabecera: "Android Doctor 1.0.0" + "Unity 6000.3.2f1 · Android · target
API 36"
├─ Toolbar: [Checks] [Gradle log]  |  [Run checks] [Scan build...] [Copy
report]
├─ Panel Checks
|   ├─ Resumen: "2 errors, 1 warning, 3 info"  (o "Nada ejecutado todavía")
|   └─ ScrollView
|       ├─ Foldout "Environment (n)"  → FindingCard...
|       ├─ Foldout "Project (n)"
|       ├─ Foldout "Native libraries (n)"
|       └─ Foldout "Build (n)"
└─ Panel Gradle log
    ├─ TextField multilínea (pegar el log)
    ├─ [Analyze pasted] [Analyze last build] [Use captured error]
    └─ Resultado: FindingCards + Foldout "Original block" + [Copy for forum]
```

`FindingCard` (un `VisualElement` que se crea a partir de un `Finding`)

- Borde izquierdo de 4 px con el color de la gravedad: error `#D9534F`, aviso `#E0A800`, info `#5B8DEF`.
- Título en negrita (`unityFontStyleAndWeight = FontStyle.Bold`) precedido del id de la regla en gris.
- Detalle con `style.whiteSpace = WhiteSpace.Normal`.
- `Foldout` «Evidence» (cerrado por defecto) con un `TextField` de solo lectura y multilínea, para poder seleccionar y copiar las rutas.
- Fila de botones solo si aplican: **Fix…** (si `FixId != null`), **Copy command** (si `Command != null`), **Docs** (si `DocsUrl != null`, con `Application.OpenURL`).

Flujo de un fix

```
var fix = FixRegistry.Get(finding.FixId);
if (fix == null) return;
if (!EditorUtility.DisplayDialog("Apply fix?", fix.Describe(finding),
"Apply", "Cancel")) return;
string message;
bool ok = fix.Apply(finding, out message);
ShowNotification(new GUIContent(message));
if (ok) RunChecks(); // vuelve a ejecutar para que el hallazgo desaparezca
```

Los fixes **recalculan lo que necesitan** con `RuleContext.Create()` y no guardan estado. Ejemplo (`Project/SetTargetApiFix.cs`):

```
using UnityEditor;

namespace AndroidDoctor.Project
{
    public sealed class SetTargetApiFix : IFix
    {
        public string Id => "FIX_TARGET_API";

        public string Describe(Finding f)
        {
            var ctx = RuleContext.Create();
            return "Target API level: " +
PlayerSettings.Android.targetSdkVersion + "  ->  " + ctx.Play.minTargetSdk;
        }

        public bool Apply(Finding f, out string message)
        {
            var ctx = RuleContext.Create();
            if (ctx.Play == null || ctx.Play.minTargetSdk <= 0) { message =
"Unknown minimum target API."; return false; }
            PlayerSettings.Android.targetSdkVersion =
(AndroidSdkVersions)ctx.Play.minTargetSdk;
            message = "Target API level set to " + ctx.Play.minTargetSdk;
            return true;
        }
    }
}
```

El cast de un entero a `AndroidSdkVersions` compila aunque tu versión de Unity no tenga un nombre para ese valor. Si Unity lo rechaza, el fix debe devolver `false` con un mensaje claro, no lanzar excepción.

Fixes de v1.0: `FIX_TARGET_API`, `FIX_MIN_API` (igual, con `PlayerSettings.Android.minSdkVersion`), `FIX_ARM64` (activa `AndroidArchitecture.ARM64` y pone IL2CPP con `PlayerSettings.SetScriptingBackend(NamedBuildTarget.Android, ScriptingImplementation.IL2CPP)`), `FIX_OPEN_EXTERNAL_TOOLS` (`SettingsService.OpenUserPreferences("Preferences/External Tools")`) y `FIX_SELECT_ASSET` (`Selection.activeObject = AssetDatabase.LoadAssetAtPath<Object>(ruta)` para enseñar un archivo en el Project). **Nunca** hay un fix que borre archivos.

Reglas de experiencia de usuario

- Nada se ejecuta solo al abrir la ventana. Se muestra «Pulsa Run checks».
- Las operaciones largas usan `EditorUtility.DisplayProgressBar(...)` dentro de `try/finally` con `ClearProgressBar()` en el `finally`, para que una excepción no deje la barra colgada.
- Si la ejecución entera falla, se muestra un solo mensaje «Android Doctor tuvo un error interno» con un botón para copiar la excepción. Nunca una ventana en blanco.
- Nunca se bloquea la ventana esperando al disco más de unos segundos: si el escaneo supera 30 s, se corta y se avisa.

Informe en Markdown (`ReportBuilder.Build(ctx, findings)`)

- Línea 1: `## Android Doctor <versión del paquete>`; la versión sale de `PackageInfo.FindForAssembly(...).version`.
- Línea 2: `Unity <versión> · <SystemInfo.operatingSystem> · target API <N o Auto> · <backend> · <arquitecturas>`.
- Línea 3: `JDK <versión> (embedded|custom) · NDK <revisión> · Gradle <versión>`; lo desconocido se escribe `unknown`.
- Después, `### Errors (n)`, `### Warnings (n)` y `### Info (n)`, cada uno con una línea `- **ID** título. detalle.` y, como máximo, las tres primeras líneas de evidencia indentadas.
- Longitud máxima 8000 caracteres; si se pasa, se corta con `… (truncated)`.
- Se copia con `EditorGUIUtility.systemCopyBuffer = texto`.
- Hay un test de «archivo dorado» (ver Tests) que fija el formato.

Tests

Estrategia en tres capas

Capa	Qué cubre	Cómo
Unitarios (Edit Mode, NUnit)	`ElfReader`, `UnityVersion`, comparación de versiones, `GradleLogParser`, `GradleTranslator`, `ReportBuilder`, reglas puras con carpetas temporales	`Window > General > Test Runner > EditMode`
Reglas que leen Player Settings	PRJ001–PRJ004, PRJ008, PRJ009	El test cambia el ajuste, ejecuta la regla y **restaura el valor original en `TearDown`**
Manuales (matriz de escenarios)	Todo lo que depende de un SDK, un build o un `.so` real	Tabla de abajo

Ayudante `Tests/Editor/ElfBuilder.cs`: genera ELFs sintéticos en memoria, así los tests de `ElfReader` no necesitan binarios. El orden de los campos de la cabecera ELF64 es el de la tabla de la sección del escáner.

```
using System.IO;

namespace AndroidDoctor.Tests
{
    public static class ElfBuilder
    {
        public sealed class Seg { public uint Type; public ulong VAddr,
MemSize, Align; }

        public const uint PT_LOAD = 1, PT_GNU_RELRO = 0x6474E552;

        public static byte[] Elf64(ushort machine, params Seg[] segs)
        {
            using (var ms = new MemoryStream())
            using (var w = new BinaryWriter(ms))
            {
                w.Write(new byte[] { 0x7F, (byte)'E', (byte)'L', (byte)'F',
2, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 }); // e_ident
                w.Write((ushort)3);            // e_type = ET_DYN
(0x10)
                w.Write(machine);              // e_machine
(0x12)
                w.Write(1u);                   // e_version
(0x14)
                w.Write(0UL);                  // e_entry
```

```
(0x18)
                w.Write(64UL);                  // e_phoff
(0x20)
                w.Write(0UL);                   // e_shoff
(0x28)
                w.Write(0u);                    // e_flags
(0x30)
                w.Write((ushort)64);            // e_ehsize
(0x34)
                w.Write((ushort)56);            // e_phentsize
(0x36)
                w.Write((ushort)segs.Length);  // e_phnum
(0x38)
                w.Write((ushort)0); w.Write((ushort)0); w.Write((ushort)0);
// shentsize, shnum, shstrndx
                foreach (var s in segs)
                {
                    w.Write(s.Type); w.Write(5u);                        //
p_type, p_flags
                    w.Write(0UL); w.Write(s.VAddr); w.Write(s.VAddr);   //
p_offset, p_vaddr, p_paddr
                    w.Write(s.MemSize); w.Write(s.MemSize); w.Write(s.Align);
// p_filesz, p_memsz, p_align
                }
                return ms.ToArray();
            }
        }
    }
}
```

`Elf32` se escribe igual con campos de 4 bytes: cabecera de 52 bytes (`e_phoff` = 52 en el offset 0x1C, `e_phentsize` = 32 en 0x2A, `e_phnum` en 0x2C) y program headers de 32 bytes en el orden `p_type, p_offset, p_vaddr, p_paddr, p_filesz, p_memsz, p_flags, p_align`.

Tests mínimos de `ElfReader` (todos obligatorios)

```
using System.IO;
using NUnit.Framework;
using AndroidDoctor.Native;

namespace AndroidDoctor.Tests
{
    public class ElfReaderTests
    {
        static ElfInfo Read(byte[] bytes)
```

```
        {
            ElfInfo info; string err;
            Assert.IsTrue(ElfReader.TryRead(new MemoryStream(bytes), out
info, out err), err);
            return info;
        }

        [Test] public void Align16K_IsAccepted()
        {
            var info = Read(ElfBuilder.Elf64(0xB7, new ElfBuilder.Seg { Type
= ElfBuilder.PT_LOAD, MemSize = 0x4000, Align = 0x4000 }));
            Assert.IsTrue(info.Is64);
            Assert.IsTrue(info.Is16KbLoadAligned);
        }

        [Test] public void Align4K_IsRejected()
        {
            var info = Read(ElfBuilder.Elf64(0xB7, new ElfBuilder.Seg { Type
= ElfBuilder.PT_LOAD, MemSize = 0x1000, Align = 0x1000 }));
            Assert.IsFalse(info.Is16KbLoadAligned);
        }

        [Test] public void OneBadLoadAmongGoodOnes_IsRejected()
        {
            var info = Read(ElfBuilder.Elf64(0xB7,
                new ElfBuilder.Seg { Type = ElfBuilder.PT_LOAD, MemSize =
0x4000, Align = 0x4000 },
                new ElfBuilder.Seg { Type = ElfBuilder.PT_LOAD, MemSize =
0x1000, Align = 0x1000 }));
            Assert.AreEqual(0x1000UL, info.MinLoadAlign());
        }

        [Test] public void MisalignedRelro_IsDetected()
        {
            var info = Read(ElfBuilder.Elf64(0xB7,
                new ElfBuilder.Seg { Type = ElfBuilder.PT_LOAD, MemSize =
0x4000, Align = 0x4000 },
                new ElfBuilder.Seg { Type = ElfBuilder.PT_GNU_RELRO, VAddr =
0x1000, MemSize = 0x800, Align = 1 }));
            Assert.IsFalse(info.IsRelroAligned16Kb);
        }

        [Test] public void NotElf_FailsWithoutThrowing()
        {
            ElfInfo info; string err;
            Assert.IsFalse(ElfReader.TryRead(new MemoryStream(new byte[] { 1,
```

```
2, 3 }), out info, out err));
            Assert.IsNull(info);
        }
    }
}
```

Tests del traductor de Gradle

- Carpeta `Tests/Editor/Fixtures/Gradle/` con un archivo por caso, **copiado de un log real**: `GRD002_1.txt`, `GRD002_2.txt`, … El prefijo es el id esperado.
- Un test con `[TestCaseSource]` que enumera los archivos, ejecuta `GradleLogParser.Extract` + `GradleTranslator.Translate` y exige que entre los hallazgos esté el id del prefijo.
- Fixtures negativos `CLEAN_1.txt` (log de un build correcto) y `UNKNOWN_1.txt` (un fallo no cubierto): el primero no devuelve nada y el segundo solo `GRD000`.
- Ruta de las fixtures: `Path.Combine(PackageInfo.FindForAssembly(typeof(ElfBuilder).Assembly).resolvedPath, "Tests/Editor/Fixtures/Gradle")`.
- Cómo provocar un log real para cada patrón está en la matriz manual (propuestas mías, no probadas; el texto que sirva de fixture se copia del log obtenido, nunca de este documento).

Tests de reglas con carpetas temporales (patrón): crear una carpeta con `Path.Combine(Path.GetTempPath(), Guid.NewGuid().ToString())`, escribir dentro lo que la regla lee (por ejemplo un archivo `release` con `JAVA_VERSION="21.0.2"` para ENV002), construir un `RuleContext` a mano con `Tools.Jdk = new ToolPath { Path = carpeta, Source = ToolSource.Custom }` y `Compat` con `jdk = 17`, ejecutar la regla y comprobar el `Severity`. Borrar la carpeta en `TearDown`.

Test de archivo dorado del informe: construir una lista fija de `Finding`, llamar a `ReportBuilder.Build` con un `RuleContext` falso y comparar con `Tests/Editor/Fixtures/report_golden.md`. Si el formato cambia a propósito, se regenera el archivo en el mismo commit.

Matriz manual (Windows y macOS × Unity 2022.3 LTS, 6000.0 LTS y la última 6.x). Son escenarios que propongo; el resultado esperado es lo que debe mostrar la herramienta.

Escenario	Cómo crearlo	Esperado
Proyecto limpio con herramientas de Unity	Proyecto nuevo, módulo Android instalado	0 errores

Escenario	Cómo crearlo	Esperado
Editor sin módulo Android	Instalación de Unity sin ese módulo	ENV001 y ninguna excepción en la Consola
JDK propio de otra versión	Instalar un JDK distinto y desmarcar «JDK Installed with Unity»	ENV002 (W); el build puede dar GRD001
Falta una plataforma del SDK	Renombrar `<sdk>/platforms/android-N`	ENV004 (E); el build da GRD003
Librería de 4 KB	Compilar `libtest4k.so` con el NDK de Unity: `aarch64-linux-android26-clang -shared -o libtest4k.so test.c -Wl,-z,max-page-size=4096`, copiarla a `Assets/Plugins/Android/arm64-v8a/` y dejar solo Android marcado en su importer	NAT001 con el dueño correcto; tras el build, BLD001
Librería de 16 KB	Igual con `-Wl,-z,max-page-size=16384`	Ningún hallazgo
Plugin de Linux de escritorio	Una `.so` en `Assets/Plugins/x86_64/` con Android desmarcado	**Ningún hallazgo** (prueba del falso positivo)
Solo 32 bits	`.so` solo en `armeabi-v7a`	NAT003
Duplicados	`foo-1.0.0.aar` y `foo-1.1.0.aar` en `Assets/Plugins/Android`	PRJ007
Manifest sin `android:exported`	Un `<activity>` con `intent-filter` y sin el atributo, target API ≥ 31	PRJ006
Gradle sin memoria	`org.gradle.jvmargs=-Xmx64M` en `gradleTemplate.properties` personalizada	El build falla; GRD009
Plantilla rota	Un error de sintaxis en `mainTemplate.gradle`	El build falla; GRD014
Rendimiento	Proyecto con más de 5000 archivos en `Assets/`	Run checks en menos de 10 s

Plan por fases

Orden obligatorio. Las horas son estimaciones mías para una persona con experiencia en Unity; con una IA ejecutora y revisión humana, cuenta más. En total, entre 45 y 60 horas, unas 2–3 semanas a tiempo parcial.

Fase	Resultado	Horas	Depende de
0	Proyecto, paquete y repo	1–2	–
1	Núcleo, runner y ventana mínima	3–4	0
2	Reglas de entorno (ENV001–009)	6–8	1
3	Reglas de proyecto (PRJ001–009) y fixes	6–8	1
4	Lector ELF y reglas NAT001–003	6–8	1
5	Paquetes conocidos, escáner de build y BLD001	4–6	4
6	Traductor de errores de Gradle	6–8	1
7	Interfaz completa e informe	6–8	2–6
8	Documentación, empaquetado y publicación	4–6	7
9	Opcional: BLD002, CI, extras	variable	8

Las fases 2, 3, 4 y 6 son independientes entre sí una vez hecha la 1.

Definición de «hecho» para toda fase

- Compila sin errores ni avisos en Unity 2022.3 LTS y en 6000.0 LTS.
- Todos los tests pasan y los de la fase están escritos.
- La Consola no muestra excepciones al abrir la ventana ni al ejecutar las reglas.
- Se cumple el criterio de aceptación de la fase.
- Un commit por tarea con mensaje Conventional Commits; al terminar, fusión a `main` y etiqueta `v0.N.0`.

Prohibido en todo el proyecto

- `record`, `init`, `async/await` que toque APIs de Unity, `System.Text.Json`, Newtonsoft y cualquier dependencia externa.
- Llamadas de red de cualquier tipo.
- Escribir en disco desde una regla.
- Inventar nombres de API de Unity. Si dudas, búsca en la documentación oficial o usa reflexión con salida segura.

- Seguir adelante cuando falla un test.

Fase 1: núcleo y ventana mínima (rama `feat/fase-1-core`)

- [ ] Crear en `Editor/Core/` los archivos de la sección «Modelo de datos y motor de reglas»: `Severity.cs`, `Finding.cs`, `IRule.cs` (con `IFix`), `UnityVersion.cs`, `RuleRunner.cs`, `Messages.cs`, `RuleContext.cs`, y las clases de datos `CompatTable.cs` y `PlayRequirements.cs` con sus JSON en `Editor/Data/`.
- [ ] Crear `Editor/Environment/AndroidTools.cs` con `AndroidToolsReader`.
- [ ] Crear `Editor/Project/DefaultApplicationIdRule.cs` (PRJ009) tal como está en el ejemplo.
- [ ] Crear `Editor/UI/AndroidDoctorWindow.cs` mínima: menú `Window/Android Doctor`, un botón *Run checks* y un `Label` por hallazgo con `RuleId` y `Title`.
- [ ] Tests: `UnityVersion.Parse("6000.3.2f1")` da 6000/3/2; `Parse("basura")` da 0/0/0; `AtLeast`; y `RuleRunner.RunAll` con una regla de prueba que lanza una excepción devuelve un `Finding` Info y sigue con las demás.

- **Aceptación:** la ventana se abre; con el identificador `com.DefaultCompany.android-doctor` aparece PRJ009 y con otro no; al menos 5 tests en verde; sin avisos en la Consola.
- **Commit:** `feat(core): finding model, rule runner and minimal window`.

Fase 2: reglas de entorno (rama `feat/fase-2-entorno`)

- [ ] Una clase por regla ENV001–ENV009 en `Editor/Environment/`, siguiendo el catálogo. Cada una usa `ctx.Tools`, `ctx.Compat` y `Messages`.
- [ ] Completar `compat.json` leyendo la documentación de Unity de cada versión; lo que no se confirme queda como `null` / `-1`.
- [ ] Lectores auxiliares puros y testeables: `JdkRelease.ReadVersion(carpeta)` (lee `<jdk>/release`), `NdkProperties.ReadRevision(carpeta)` (`source.properties`), `GradleVersion.FromLauncherJar(carpeta)`.
- [ ] Tests con carpetas temporales para ENV002, ENV003, ENV004, ENV006, ENV007 y ENV008 (en el último, poner la variable con `Environment.SetEnvironmentVariable` y restaurarla en `TearDown`).

- **Aceptación:** con las herramientas de Unity, 0 errores de entorno; al renombrar `platforms/android-N` aparece ENV004; en un editor sin módulo Android aparece ENV001 y no hay excepciones.
- **Commit:** `feat(env): environment rules ENV001-ENV009`.

Fase 3: reglas de proyecto y fixes (rama `feat/fase-3-proyecto`)

- [ ] Una clase por regla PRJ001–PRJ008 en `Editor/Project/` (PRJ009 ya existe).
- [ ] `ArtifactName.Parse("play-services-basement-18.3.0.aar")` devuelve `("play-services-basement", "18.3.0")`; con tests para nombres sin versión, con sufijos y `.jar`. PRJ007 lo usa para agrupar.
- [ ] `ManifestExportedChecker` puro: recibe el texto de un manifest y devuelve la lista de elementos sin `android:exported`; con manifests de ejemplo en `Fixtures/`.
- [ ] Fixes `FIX_TARGET_API`, `FIX_MIN_API`, `FIX_ARM64`, `FIX_OPEN_EXTERNAL_TOOLS`, `FIX_SELECT_ASSET` y `FixRegistry`.

- **Aceptación:** al cambiar Player Settings aparecen y desaparecen los hallazgos; cada fix pide confirmación y muestra «antes → después»; los tests de PRJ restauran los Player Settings.
- **Commit:** `feat(project): project rules PRJ001-PRJ009 and fixes`.

Fase 4: lector ELF y librerías nativas (rama `feat/fase-4-elf`)

- [ ] `Native/ElfReader.cs` exactamente como en el documento, y `Tests/Editor/ElfBuilder.cs` con los 5 tests mínimos más uno de ELF32.
- [ ] `Native/NativeLibScanner.cs` con el filtro `PluginImporter` y el cálculo de dueño.
- [ ] Reglas NAT001, NAT002 y NAT003 que usan `ctx.GetOrCompute("nativeLibs", ...)`.
- [ ] Probar a mano con las librerías reales de 4 KB y 16 KB de la matriz.

- **Aceptación:** los tests pasan; la librería de 4 KB da NAT001 con el dueño correcto, la de 16 KB nada, y el plugin de Linux de escritorio nada.
- **Commit:** `feat(native): ELF reader and 16 KB alignment rules`.

Fase 5: paquetes conocidos y build final (rama `feat/fase-5-build`)

- [ ] `VersionCompare.IsLess(a, b)`: compara solo la parte numérica antes de cualquier `-` (la versión `1.9.0-pre.1` se trata como `1.9.0`); con tests.
- [ ] Reglas NAT004 y NAT005 a partir de `play.json.knownPackages`.
- [ ] `Build/ArtifactScanner.cs` (como en el documento) y `Build/PostBuildScan.cs`.
- [ ] BLD001 y botón *Scan build…* (`EditorUtility.OpenFilePanel`) para analizar un `.apk` / `.aab` a mano.

- **Aceptación:** un build con la librería de 4 KB da BLD001 con el nombre del `.so`; uno limpio escribe la línea de éxito en la Consola.
- **Commit:** `feat(build): artifact scanner and known package rules`.

Fase 6: traductor de Gradle (rama `feat/fase-6-gradle`)

- [ ] `LogSource`, `GradleLogParser`, `GradlePatterns` y `GradleTranslator`.
- [ ] Captura automática con `[InitializeOnLoad]` y `Application.logMessageReceived`.
- [ ] Provocar y guardar en `Fixtures/Gradle/` logs reales de, como mínimo, GRD002, GRD003, GRD009 y GRD014 con las recetas de la matriz; para los demás patrones, buscar en los foros de Unity logs reales y copiarlos.
- [ ] Un patrón sin fixture real se queda fuera de v1.0.

- **Aceptación:** todas las fixtures pasan; `CLEAN_1` no da nada y `UNKNOWN_1` da solo GRD000; tras un fallo forzado, *Analyze last build* encuentra el bloque.
- **Commit:** `feat(gradle): log parser and error translator`.

Fase 7: interfaz completa e informe (rama `feat/fase-7-ui`)

- [ ] `FindingCard`, foldouts por categoría, pestaña *Gradle log* y barra de progreso con `try/finally`.
- [ ] `ReportBuilder` y botones *Copy report* y *Copy for forum*.
- [ ] Flujo de fix con diálogo, notificación y reejecución.
- [ ] Test de archivo dorado del informe.

- **Aceptación:** los ocho pasos de «Funcionamiento final esperado» funcionan de principio a fin en Windows y macOS.
- **Commit:** `feat(ui): full window, report export and fix flow`.

Fase 8: documentación y publicación (rama `chore/release-1.0`)

- [ ] README, CHANGELOG, capturas y GIF, versión `1.0.0` en `package.json`, etiqueta `v1.0.0` y página de itch.io, como se detalla en la sección «Publicación».
- [ ] Instalar el paquete desde la URL de git en un proyecto limpio y comprobar que todo funciona.

- **Aceptación:** instalación limpia por URL de git, sin errores; la lista de comprobación de lanzamiento está completa.

Fase 9: opcional

- [ ] BLD002 (alineación ZIP): leer el directorio central del zip a mano para obtener el desplazamiento de cada entrada sin comprimir y comprobar que es múltiplo de 16384; solo informativo.
- [ ] CI con GitHub Actions: necesita activar una licencia de Unity en el runner, así que se deja para después de v1.0.
- [ ] Traducciones adicionales en `Messages`.

Publicación

Antes de publicar: comprueba que el nombre «Android Doctor» no choca con un producto existente en itch.io, GitHub, la Asset Store ni OpenUPM. No lo he comprobado. Si choca, cambia el nombre en `package.json`, README y página antes del primer post.

README.md (estructura fija)

1. Título y una frase: «Finds why your Unity Android build fails or gets rejected by Google Play, and tells you how to fix it.»
2. Un GIF de menos de 10 s: *Run checks*, tarjetas de error, un fix.
3. Instalación: `Window > Package Manager > + > Add package from git URL…` con la URL `https://github.com/<usuario>/android-doctor.git?path=/Packages/com.<usuario>.androiddoctor`.
4. Qué comprueba (la tabla del catálogo, resumida).
5. Cómo leer un hallazgo y cómo reportar un bug con *Copy report*.
6. Limitaciones honestas (qué no hace, versiones de Unity soportadas).
7. Licencia MIT y cómo contribuir.

Versionado: SemVer. `CHANGELOG.md` en formato *Keep a Changelog*. Cambiar solo `package.json.version`, el changelog y la etiqueta.

Lista de comprobación de lanzamiento

- [ ] Tests en verde en 2022.3 LTS y 6000.0 LTS; matriz manual hecha en Windows y macOS.
- [ ] `package.json` en `1.0.0`; `CHANGELOG.md` con la fecha de hoy; etiqueta `v1.0.0` subida.
- [ ] Instalación por URL de git probada en un proyecto vacío.
- [ ] README con el GIF; plantilla de issue que pide el resultado de *Copy report*.
- [ ] Un `.zip` de la carpeta del paquete en GitHub Releases (para quien instale con *Add package from disk*).
- [ ] Página de itch.io creada con los mismos textos.

Página de itch.io

- Título: «Android Doctor – fix Gradle and Google Play 16 KB errors in Unity».
- Precio: pay-what-you-want con mínimo 0.
- Archivos: el `.zip` del paquete. La URL de git va en la descripción como vía principal.
- Descripción: primero el problema en dos frases, luego el GIF, luego los tres «qué hace» (entorno, 16 KB, errores de Gradle), luego la instalación.
- Etiquetas: `unity`, `tool`, `android`, `editor-tool`.

Mensaje para Reddit (en inglés; adáptalo)

```
Title: I made a free tool that tells you why your Unity Android build fails
(and which SDK breaks your 16 KB Play requirement)

Body:
Every time I (or a friend) hit "Gradle build failed", the real cause was
buried in a wall of text.
So I built Android Doctor: one window that checks your JDK/SDK/NDK/Gradle
setup, scans every
native library for 16 KB alignment (and tells you WHICH package ships the bad
one), and translates
Gradle errors into a probable cause + fix.

Free, MIT, no network calls. Works on Unity 2022.3 and 6.x.
Install via git URL: <url>
If something is not detected, press "Copy report" and paste it in an issue -
that's the most useful feedback.
```

Antes de publicar en cada subreddit, **lee sus normas de autopromoción** (no las he consultado). No publiques el mismo texto en varios a la vez y responde a los comentarios los primeros días. Otra vía legítima: cuando alguien abra un hilo real sobre 16 KB o `Gradle build failed` en Unity Discussions, responder con una solución concreta y enlazar la herramienta como una opción más, no como spam.

Soporte tras el lanzamiento

- Plantilla de issue: versión de Unity, sistema operativo, el informe de *Copy report* y qué esperabas.
- Cada falso positivo reportado se convierte en un test antes de arreglarlo.
- Cada error de Gradle nuevo que alguien traiga se convierte en un patrón con su fixture (con permiso para copiar el log).

Métricas a mirar (sin obsesión): descargas en itch.io, estrellas e issues en GitHub, y cuántos issues traen el informe adjunto.